

Проект по Конструирование программного обеспечения

За реализацию отвечают студенты группы 5130904/30108: Петухов Кирилл, Аракелян Даниил, Полелейко Иван, Скопченко Александр

Название проекта: Crypto Tracker – приложение для отслеживания криптовалют

Цель проекта:

Создать минимально жизнеспособное приложение, которое позволяет пользователю отслеживать курсы выбранных криптовалют, получать уведомления о значимых изменениях и быть в курсе актуальной информации о рынке.

Возможности приложения:

1. Просмотр популярных валют, графиков с ценой, сохранение выбранной валюты в "Избранное".
2. Уведомление в ТГ боте при изменении стоимости валюты: + или -, в процентах, процент выбираете Вы.
3. Портфель + инвестиции
 - а. Возможность добавить «виртуальный портфель» в профиле (сумма покупки + количество монет) и следить за прибылью/убытком.
 - б. Подсказки: «Ваш портфель просел на 10% за неделю, хотите выставить стоп-уведомление?».
4. Разные сценарии уведомлений: «инвестор» (редкие отчеты), «трейдер» (частые сигналы).
5. Изменение темы оформления веб приложения (темная/светлая)

6. Интеграция с телеграмм ботом (/alert BTC 0.01%) + уведомления

Реализация:

Telegram bot + Web Ui

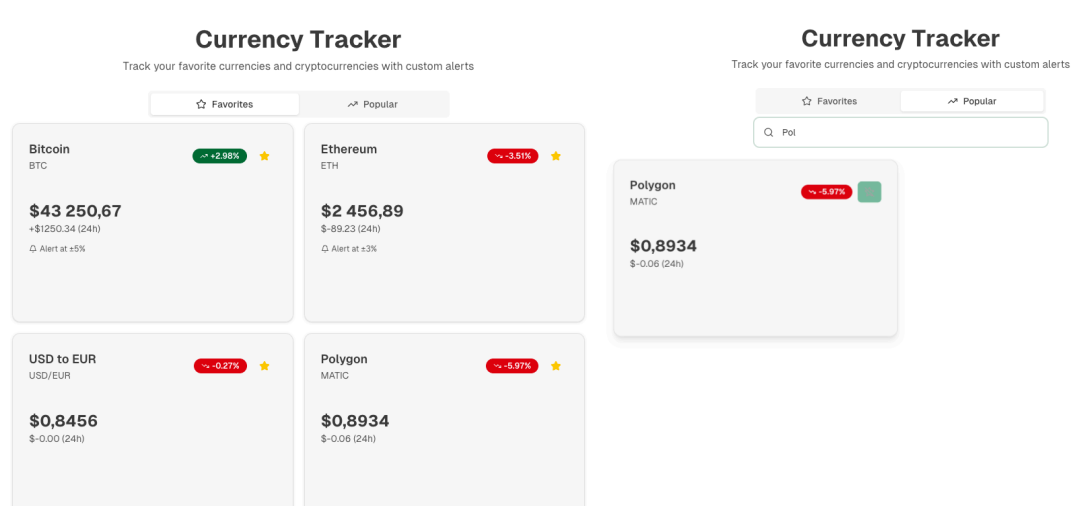
Стек:

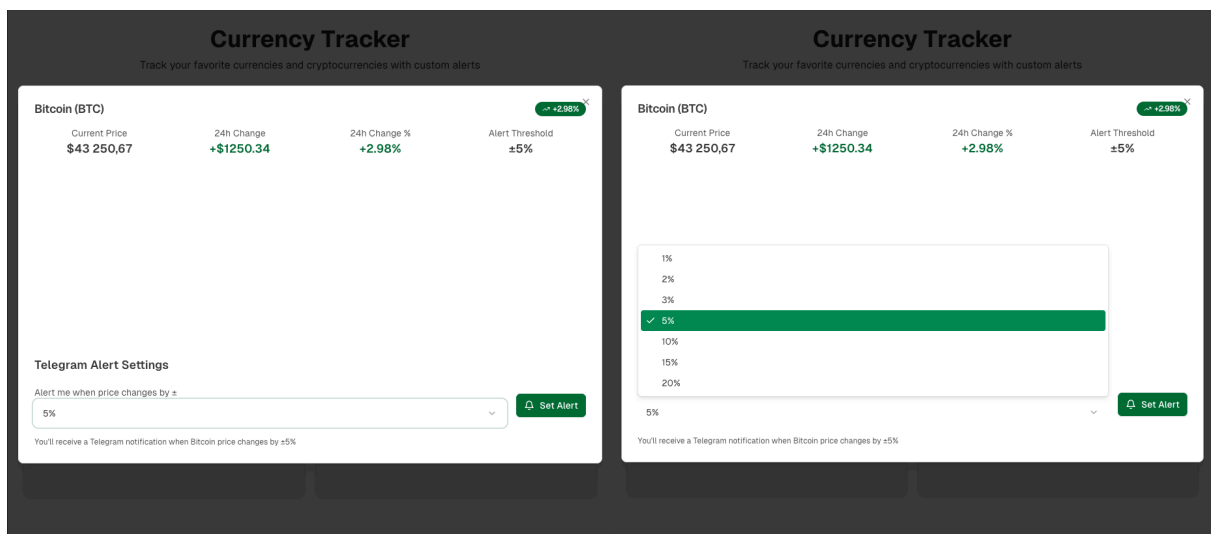
Python + FastAPI, TypeScript, PostgreSQL

Ссылка на репозиторий:

<https://github.com/CryptoTracker-pi8/crypto-tracker>

Макет:





Юзер стори

1. Как пользователь я хочу просматривать цены и графики валют.
2. Как пользователь я хочу сохранять определенные валюты, чтобы не терять их из виду.
3. Как пользователь я хочу отслеживать стоимость “Избранной” валюты. Получать уведомления в телеграмм боте при изменении стоимости валюты в определенный + или - процент.
4. Как пользователь я хочу иметь возможность взаимодействовать с приложением через команды в телеграмм боте, чтобы постоянно не открывать веб интерфейс приложения. (/status BTC, /alert BTC 0.01 (+-%) и т п)
5. Как пользователь я хочу иметь виртуальный портфель для отслеживания его состояния.
6. Как пользователь я хочу получать информацию по состоянию своего портфеля.
7. Как пользователь я хочу иметь возможность кастомизации темы веб приложения.

Выработка требований

Оценка NFR для проекта Crypto Tracker

throughput (RW QPS)

- ~10 RPS в пике (80% чтения, 20% записи)

latency (RW)

- p95 < 200 мс для API
- уведомления в Telegram до 5 сек

R/W ratio

- ~80% чтения, 20% записи

traffic volume (RW)

- ~1-2 GB/день

storage: disk, RAM

- **Postgres:** ~10 MB / 10k пользователей (данные портфелей)
- **История цен:** до 1 GB/месяц
- **RAM:** 2-4 GB

latency & throughput

- FastAPI + Postgres обеспечивают нужные показатели

CAP

- **Consistency:** strong (портфель), eventual (цены)
- **Availability:** репликация + failover

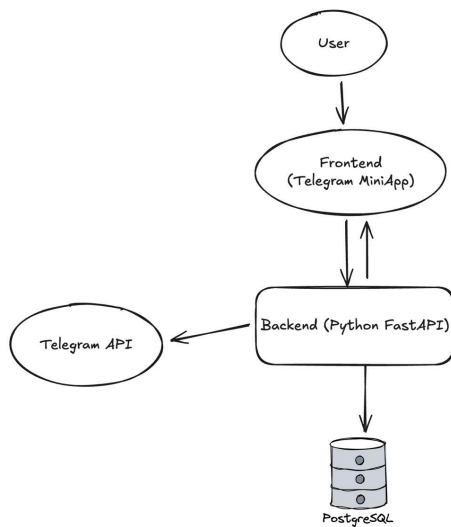
performance vs scalability

- 1 сервер (4 vCPU, 8 GB RAM) для 10k пользователей
- при росте: горизонтальное масштабирование + Redis

costs

- ~2000 руб/мес. (сервер)

Архитектура и проектировани



Auth / Пользователи

- auth_user
- auth_refresh_token
- user_settings

Favorites / Избранное

- favorite

Portfolios / Портфели

- portfolio
- lot

Alerts / Уведомления

- alert
- alert_trigger_log

Реализация

Бекенд:

Crypto Tracker 1.0.0 QA3.1

API

Микросервис для отслеживания курса криптовалют

default		^
GET	/api/v1/health-check	Healthcheck
currencies		^
GET	/api/v1/currencies	Get Currencies
GET	/api/v1/currencies/{symbol}	Get Currency
GET	/api/v1/currencies/{symbol}/history	Get Currency History
favorites		^
POST	/api/v1/favorites	Create Favorite
GET	/api/v1/favorites	Get Favorites
DELETE	/api/v1/favorites/{currency_symbol}	Delete Favorite
portfolio		^
POST	/api/v1/portfolio	Create Or Edit Portfolio
GET	/api/v1/portfolio	Get Portfolio
POST	/api/v1/portfolio/investments	Add Investment
GET	/api/v1/portfolio/stats	Get Stats
DELETE	/api/v1/portfolio/investments/{inv_id}	Delete Investment
alerts		^
POST	/api/v1/alerts	Create Alert
GET	/api/v1/alerts	List Alerts
DELETE	/api/v1/alerts/{alert_id}	Delete Alert
settings		^
GET	/api/v1/settings	Get Settings
PUT	/api/v1/settings	Update Settings

currencies		^
GET	/api/v1/currencies	Get Currencies
GET	/api/v1/currencies/{symbol}	Get Currency
GET	/api/v1/currencies/{symbol}/history	Get Currency History
favorites		^
POST	/api/v1/favorites	Create Favorite
GET	/api/v1/favorites	Get Favorites
DELETE	/api/v1/favorites/{currency_symbol}	Delete Favorite
portfolio		^
POST	/api/v1/portfolio	Create Or Edit Portfolio
GET	/api/v1/portfolio	Get Portfolio
POST	/api/v1/portfolio/investments	Add Investment
GET	/api/v1/portfolio/stats	Get Stats
DELETE	/api/v1/portfolio/investments/{inv_id}	Delete Investment
alerts		^
POST	/api/v1/alerts	Create Alert
GET	/api/v1/alerts	List Alerts
DELETE	/api/v1/alerts/{alert_id}	Delete Alert
settings		^
GET	/api/v1/settings	Get Settings
PUT	/api/v1/settings	Update Settings

Тесты:

1. Юнит тесты

```

===== tests coverage =====
----- coverage: platform darwin, python 3.13.2-final-0 -----
Name                                                    Stmts  Miss  Cover   Missing
-----
cryptotracker/_init__.py                               2      0    100%
cryptotracker/__main__.py                              27      7    74%    15-24
cryptotracker/api/_init__.py                           7      0    100%
cryptotracker/api/endpoints/alerts.py                 36      6    83%    34-35, 49-50, 69-70
cryptotracker/api/endpoints/currencies.py             43     12    72%    25-33, 45, 53-54, 71, 77-78
cryptotracker/api/endpoints/favorites.py              36      6    83%    42-43, 66-67, 93-94
cryptotracker/api/endpoints/healthcheck.py             5      1    80%    8
cryptotracker/api/endpoints/portfolio.py              67     19    72%    40-41, 59-62, 78, 81-82, 96-101, 120-123
cryptotracker/api/endpoints/settings.py              23     10    57%    23-27, 39-43
cryptotracker/api/schemas/alerts_schemas.py          24      0    100%
cryptotracker/api/schemas/currencies_schemas.py     22      0    100%
cryptotracker/api/schemas/favorites_schemas.py      15      0    100%
cryptotracker/api/schemas/portfolio_schemas.py       39      0    100%
cryptotracker/api/schemas/settings_schemas.py       13      0    100%
cryptotracker/api/services/alerts_service.py          51      1    98%    76
cryptotracker/api/services/coin_gecko_service.py     125     32    74%    30-34, 63, 71, 96, 101, 111, 115-119, 134-136, 152, 156-160, 164-171
cryptotracker/api/services/favorite_service.py        37      0    100%
cryptotracker/api/services/portfolio_service.py       91      5    95%    43-48, 78, 147-148
cryptotracker/api/services/settings_service.py        24     17    29%    14-23, 26-33
cryptotracker/background/_init__.py                   0      0    100%
cryptotracker/background/alerts_scheduler.py          74     55    26%    23-25, 32-95, 103-115, 123-125
cryptotracker/config/_init__.py                       0      0    100%
cryptotracker/config/default.py                       23      1    96%    51
cryptotracker/config/utils.py                          3      0    100%
cryptotracker/database/_init__.py                     4      0    100%
cryptotracker/database/base.py                        3      0    100%
cryptotracker/database/connection.py                  29      6    79%    49-52, 60-62
cryptotracker/database/models/_init__.py              6      0    100%
cryptotracker/database/models/alert.py               27      0    100%
cryptotracker/database/models/favorite.py            12      0    100%
cryptotracker/database/models/portfolio.py            22      0    100%
cryptotracker/database/models/user.py                 15      0    100%
cryptotracker/database/models/user_settings.py        13      0    100%
cryptotracker/domains/currencies/repository.py        18     12    33%    8-21, 24-37, 40-47
cryptotracker/utils/auth.py                           11      4    64%    20-24
TOTAL
59 passed, 8 warnings in 1.17s

```

2. Нагрузочное тестирование



```

execution: local
script: backend/scripts/load_test_k6.js
output: -

scenarios: (100.00%) 1 scenario, 50 max VUs, 5m30s max duration (incl. graceful stop):
  * default: Up to 50 looping VUs for 5m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

THRESHOLDS

http_req_duration
  ✓ 'p(95)<1500' p(95)=447.73ms

TOTAL RESULTS

checks_total.....: 68285 227.509157/s
checks_succeeded...: 99.99% 68281 out of 68285
checks_failed.....: 0.00% 4 out of 68285

✓ currencies ok
✓ currency ok
✓ history ok
✓ stats ok
✗ favorite create ok
  ↳ 99% — ✓ 9753 / ✗ 2
✓ favorites ok
✗ favorite delete ok
  ↳ 99% — ✓ 9753 / ✗ 2

HTTP
http_req_duration.....: avg=55.29ms min=312µs med=6.58ms max=1.35s p(90)=138.21ms p(95)=447.73ms
  { expected_response:true }...: avg=55.3ms min=312µs med=6.58ms max=1.35s p(90)=138.25ms p(95)=447.74ms
http_req_failed.....: 0.00% 4 out of 68285
http_reqs.....: 68285 227.509157/s

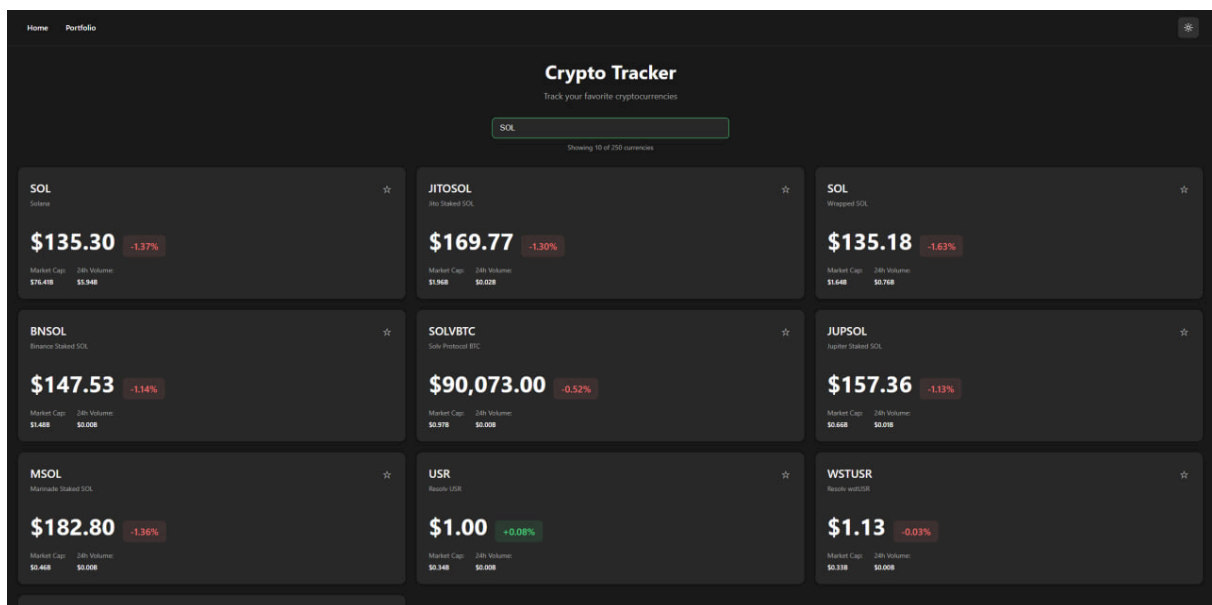
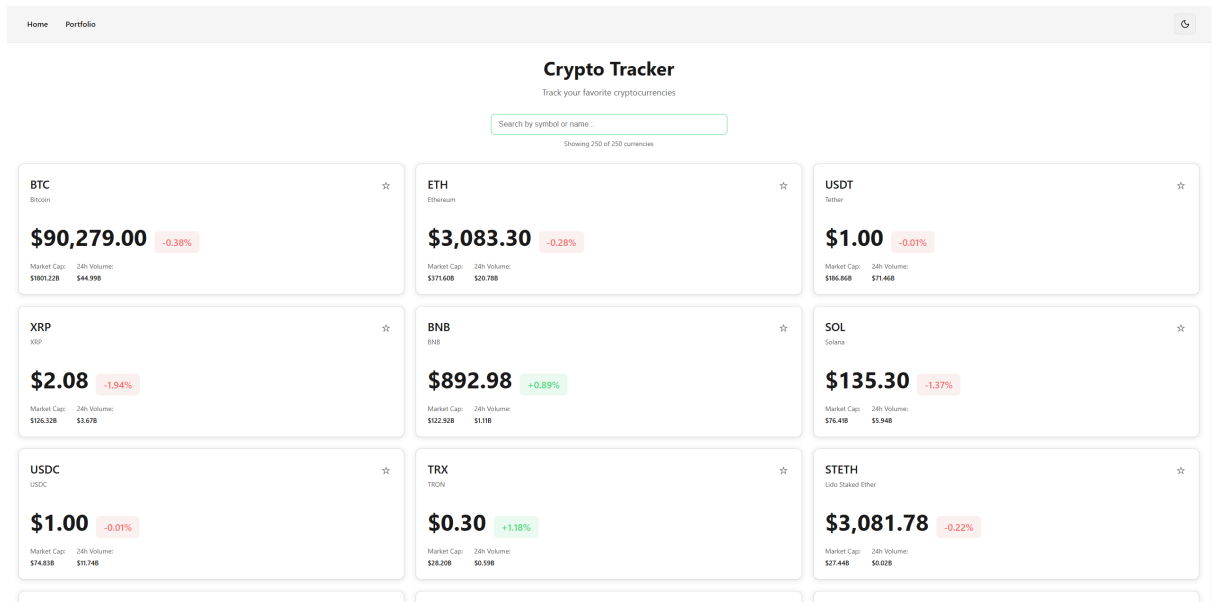
EXECUTION
iteration_duration.....: avg=1.38s min=1.01s med=1.34s max=3.02s p(90)=1.93s p(95)=2.06s
iterations.....: 9755 32.501308/s
vus.....: 1 min=1 max=50
vus_max.....: 50 min=50 max=50

NETWORK
data_received.....: 205 MB 683 kB/s
data_sent.....: 8.2 MB 27 kB/s

running (5m00.1s), 00/50 VUs, 9755 complete and 0 interrupted iterations
default ✓ [=====] 00/50 VUs 5m0s

```

Фронтенд:



← Back

Bitcoin

BTC

\$90,285.00

+0.47% 24h

Market Cap

\$1802.50B

24h Volume

\$43.47B

Price History

7d 30d 90d



← Back

Bitcoin

BTC

\$90,285.00

+0.47% 24h

Market Cap

\$1802.50B

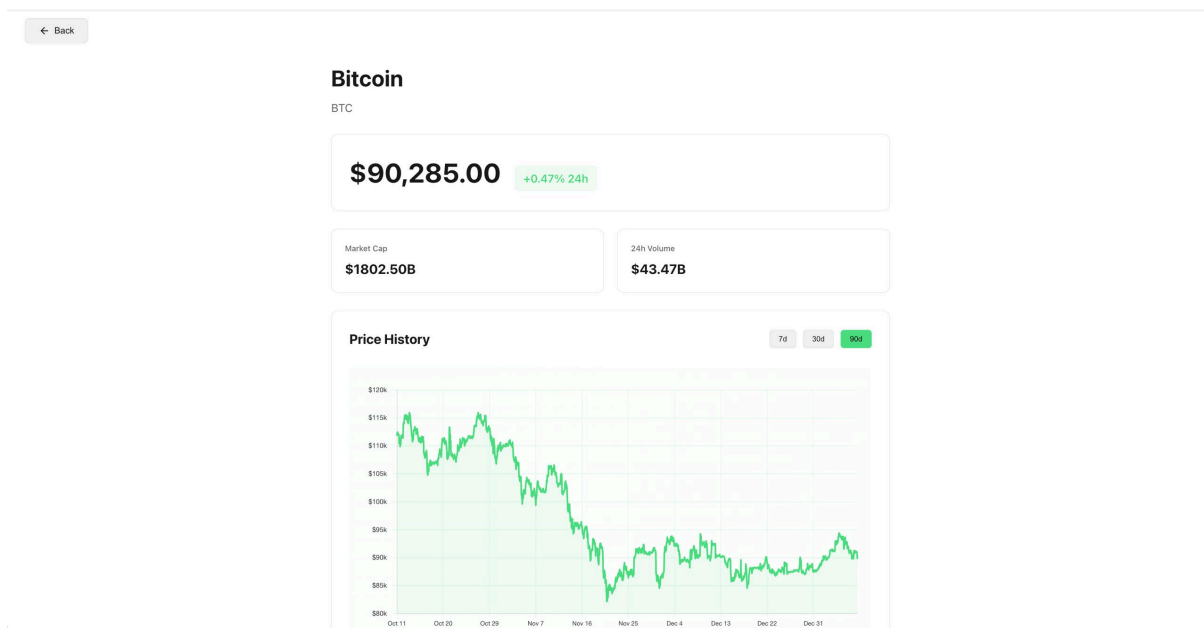
24h Volume

\$43.47B

Price History

7d 30d 90d





Home Portfolio

Create Portfolio

Save your holdings and track P&L in one place.

Existing Portfolio Remove portfolio Refresh

test Investments: 1

Total invested \$20.00	Current value \$180,518.00	P&L \$180,518.00 +902,598.00%
----------------------------------	--------------------------------------	---

Holdings

Symbol	Amount	Buy price
BTC	2,000,000.00	\$10.00

Add holding

Symbol
BTC

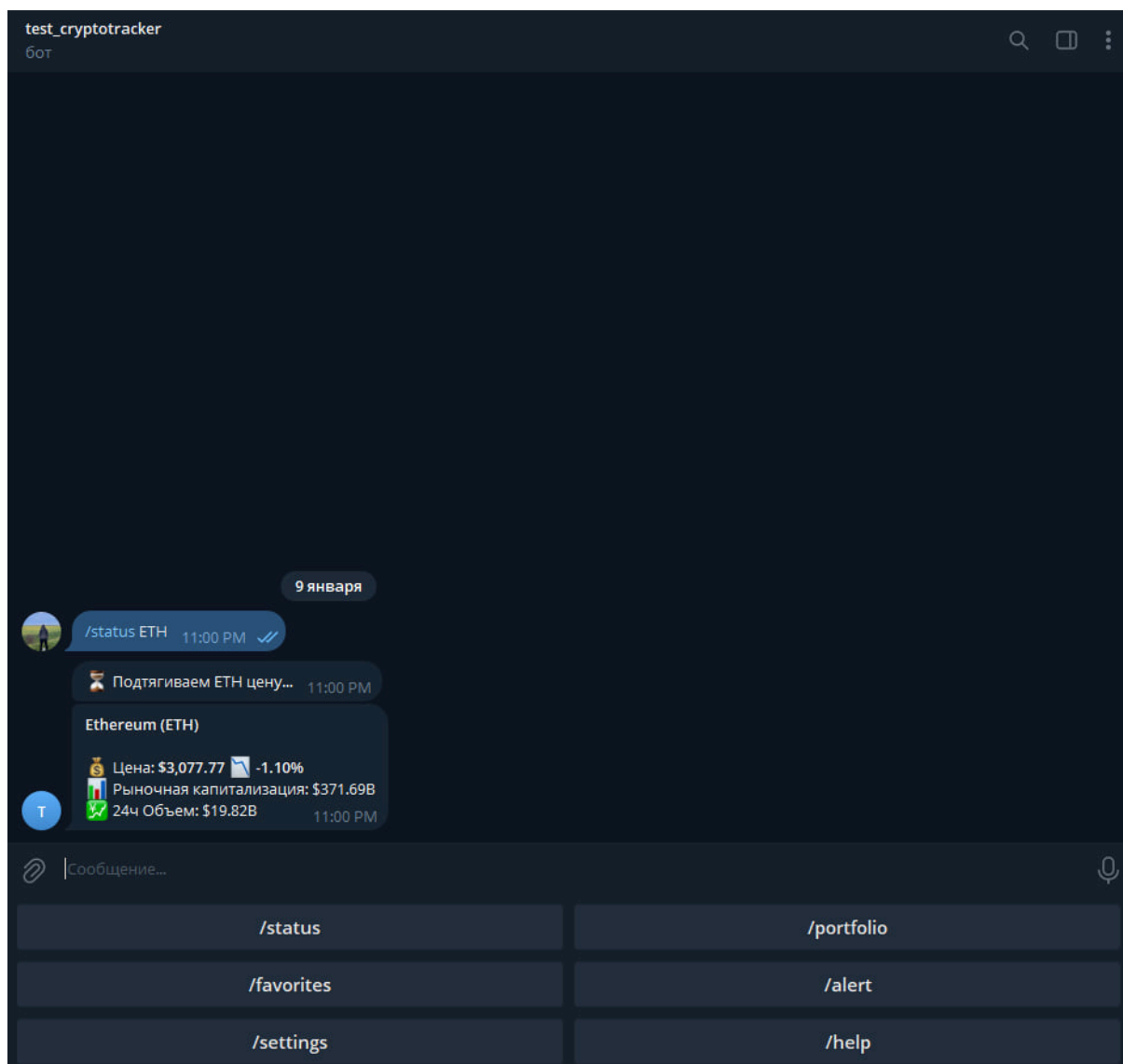
Amount
0.1

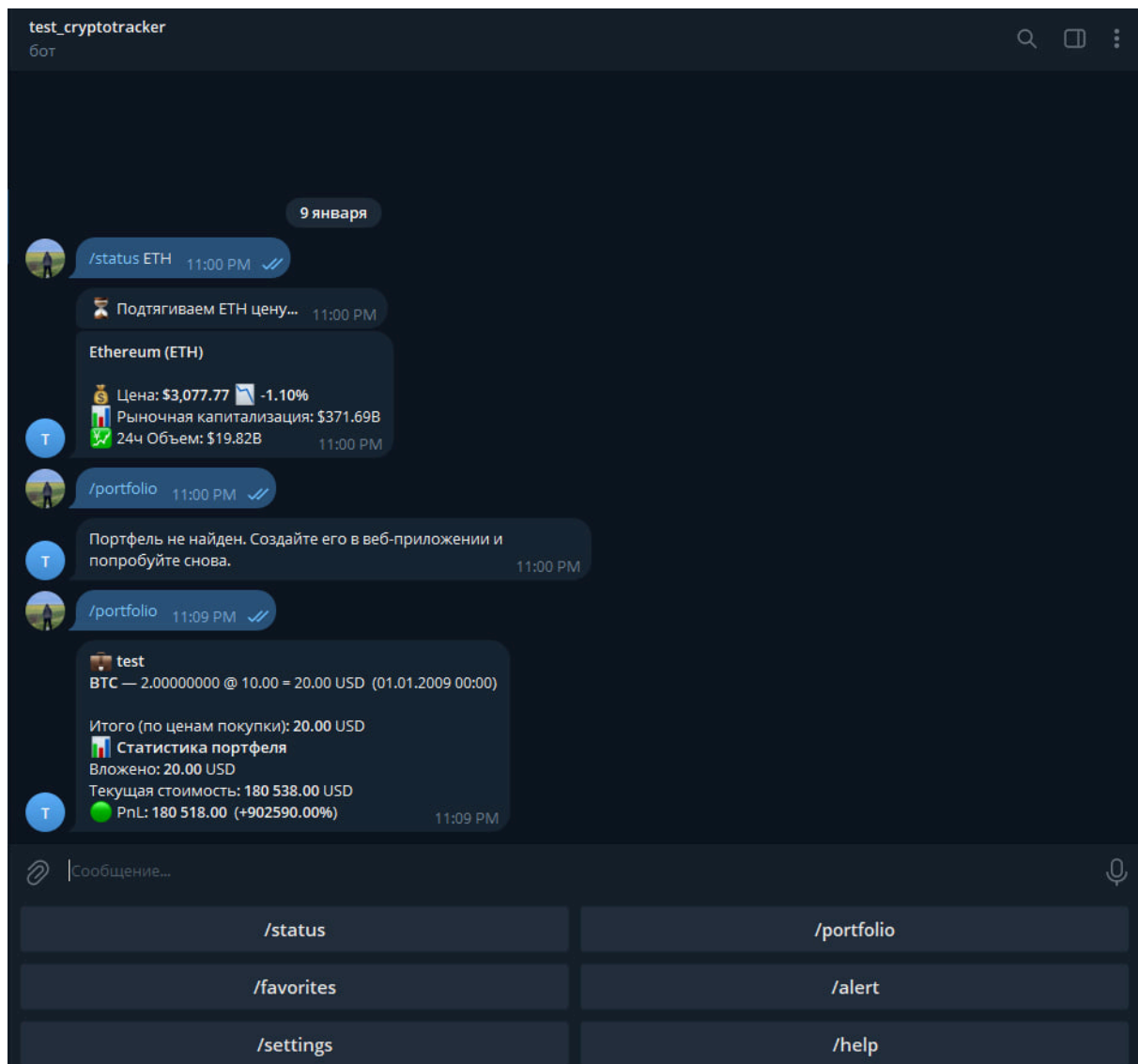
Buy price (USD)
20000

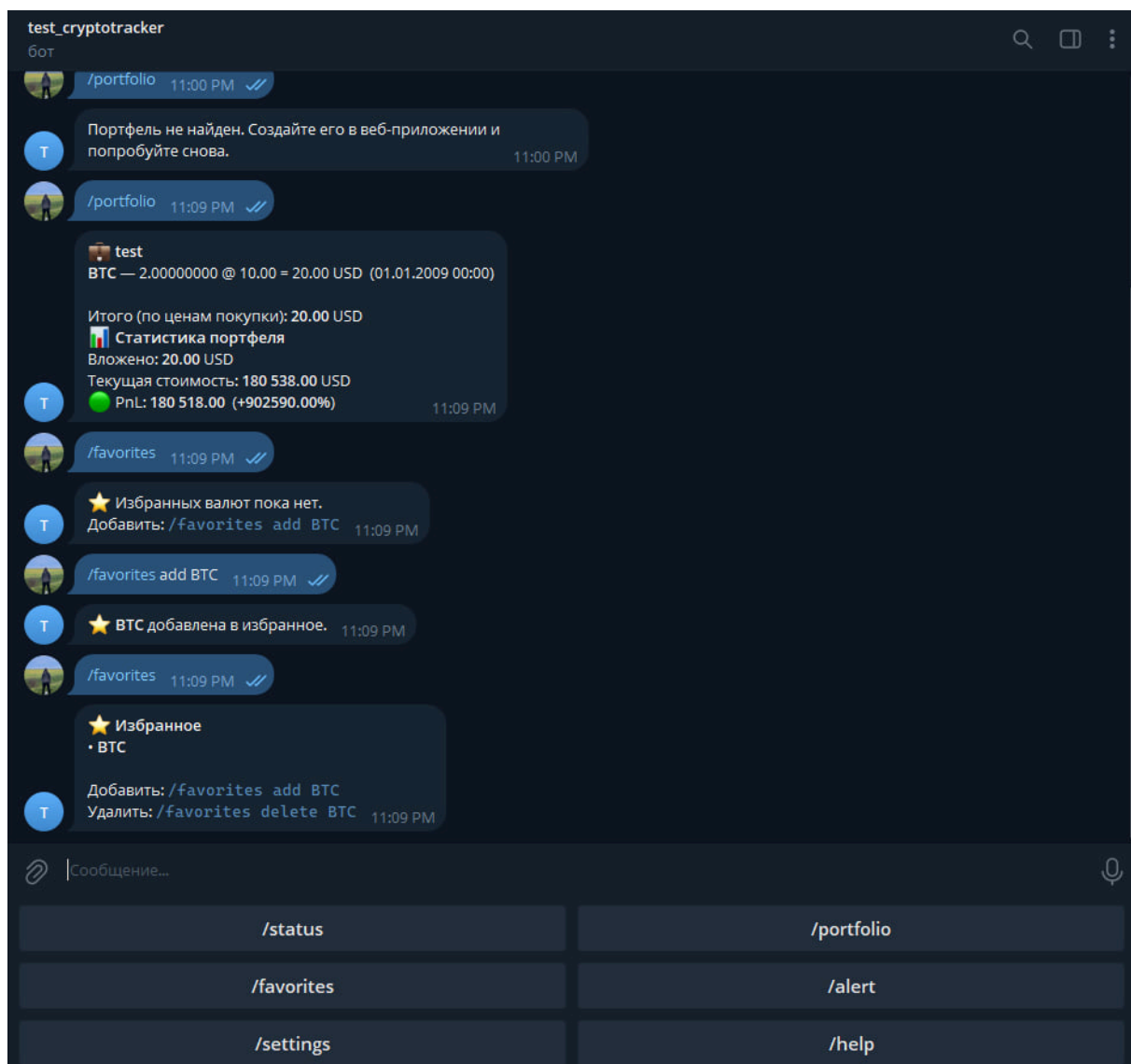
Bought at
Jul 29, 2017

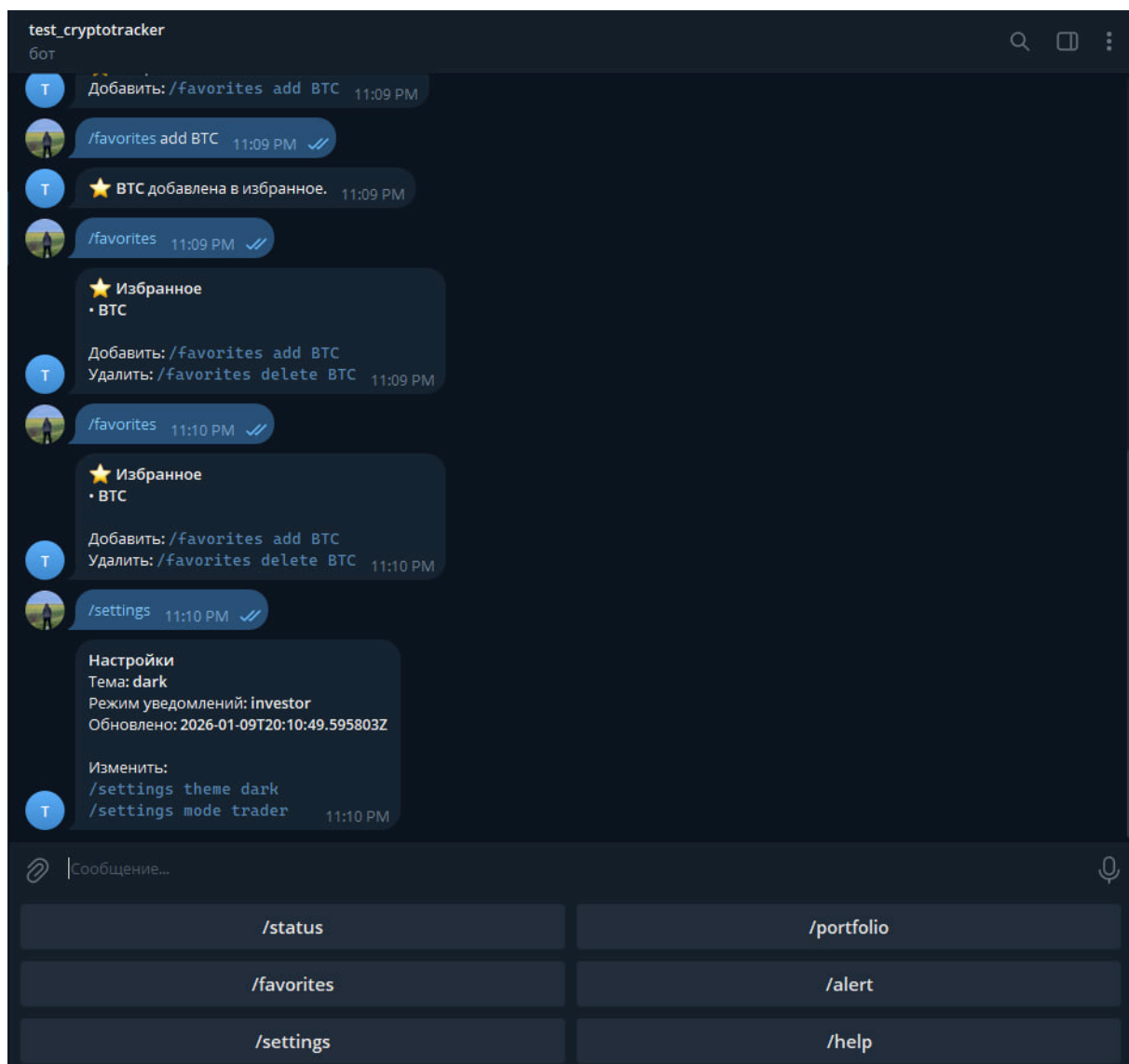
Add holding

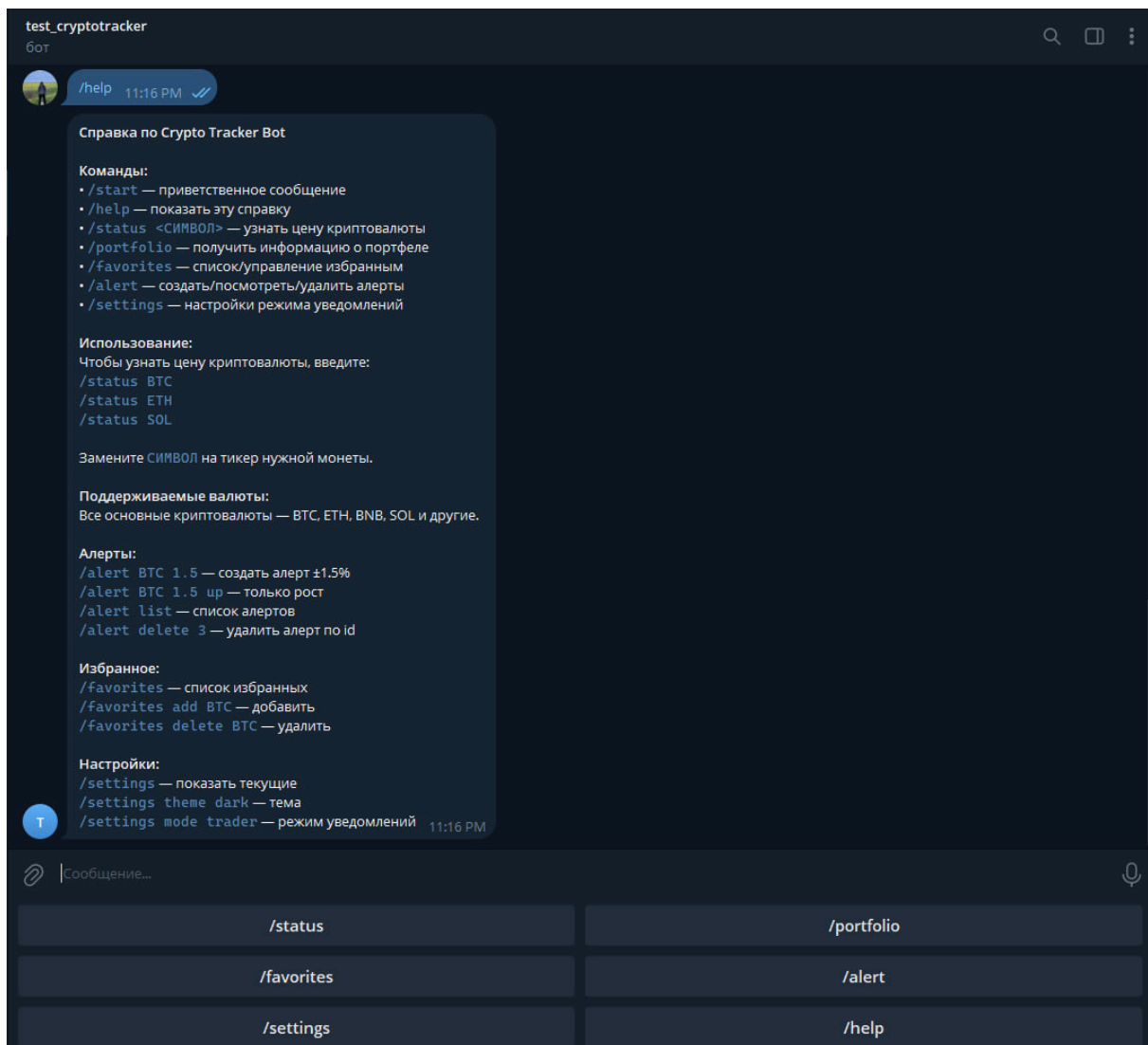
Телеграм бот:

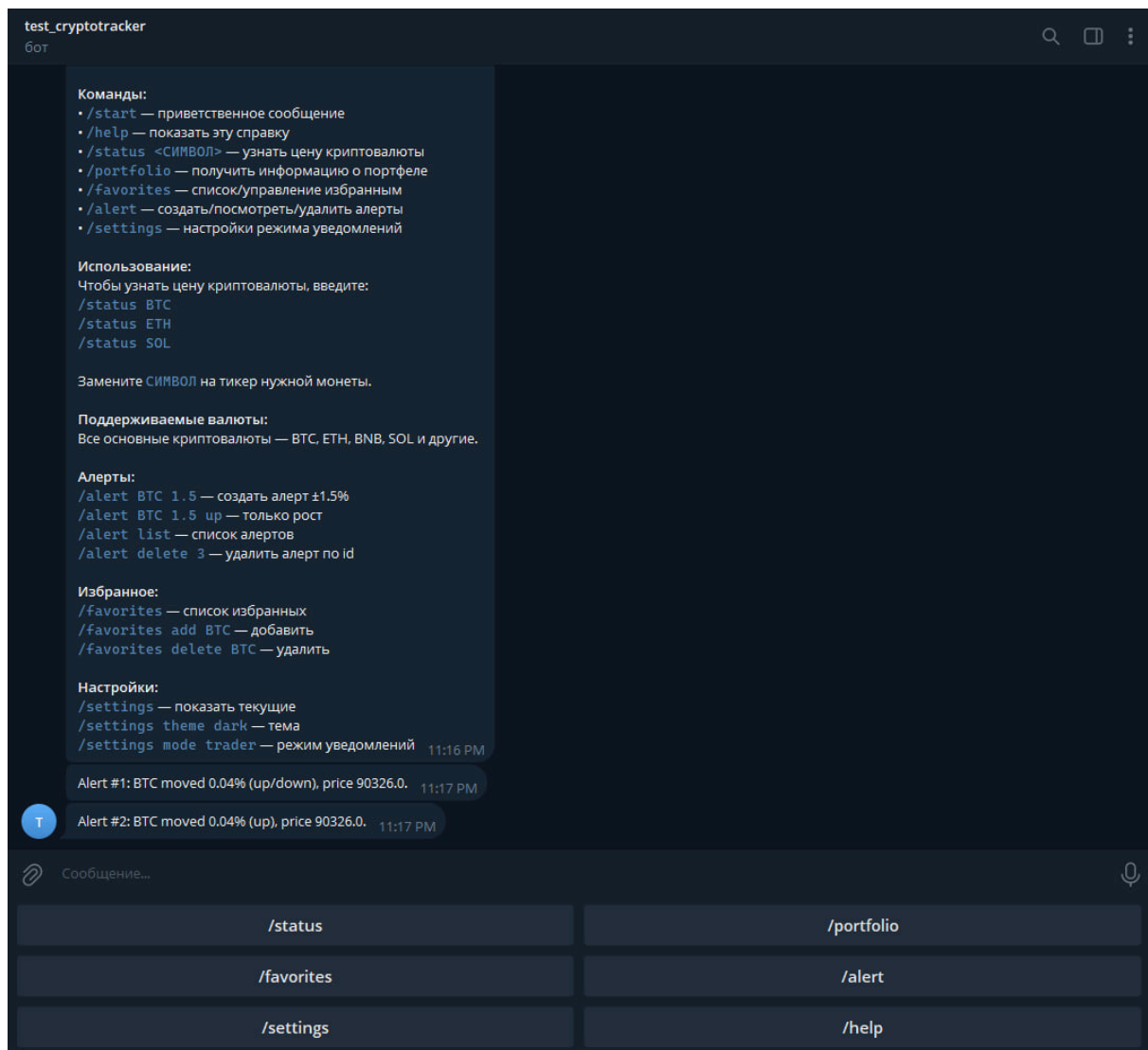


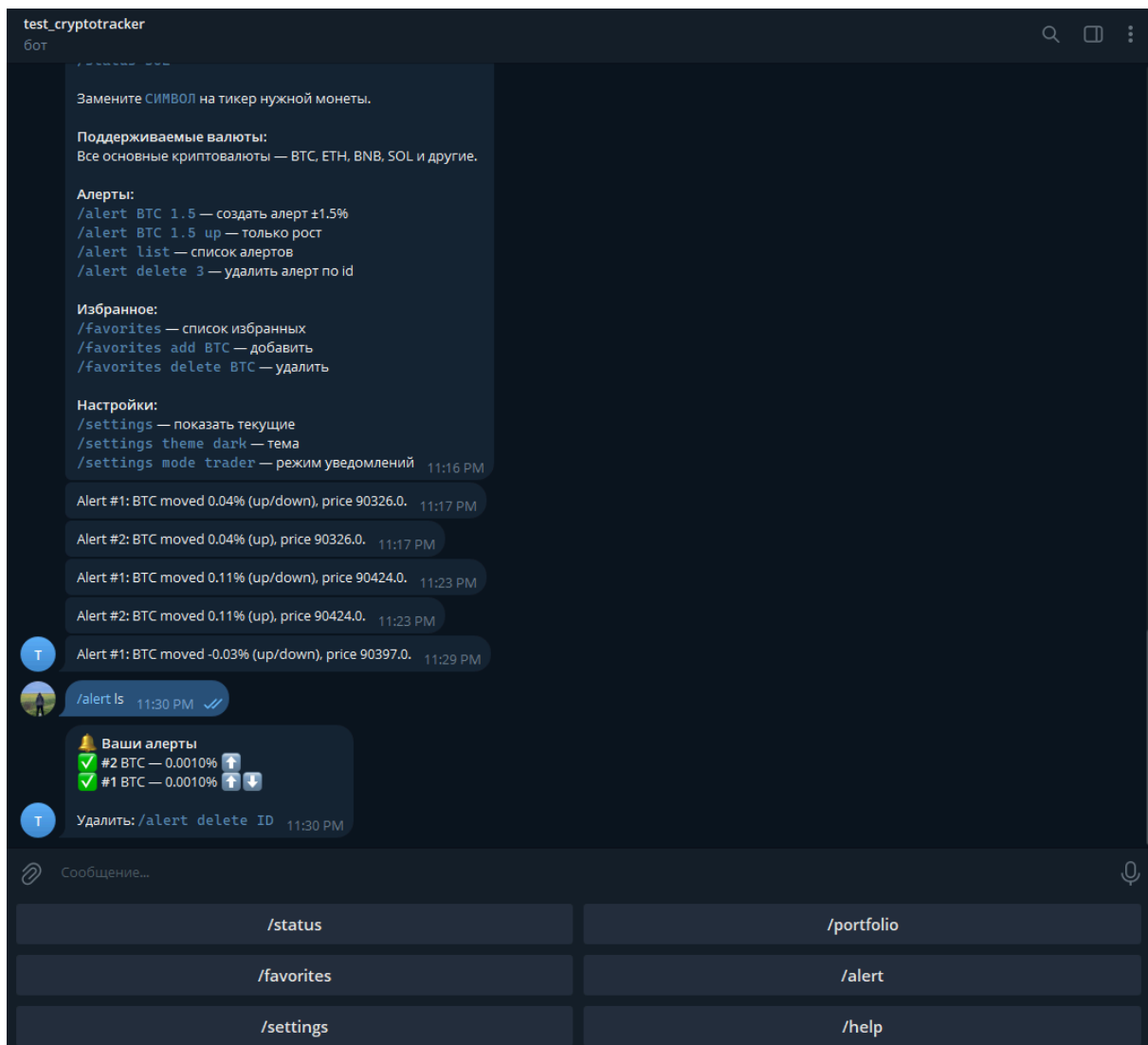












Сборка

Запуск проекта осуществляется через docker-compose файл

```
# /bot/Dockerfile
```

```
FROM python:3.13-slim
```

```
ENV PYTHONDONTWRITEBYTECODE=1
```

```
ENV PYTHONUNBUFFERED=1
```

```
WORKDIR /app
```

```
# Copy project metadata and source
COPY pyproject.toml uv.lock README.md ./
COPY cryptotracker_bot ./cryptotracker_bot

# Install dependencies declared in pyproject.toml
RUN pip install --no-cache-dir --upgrade pip \
    && pip install --no-cache-dir .

# Run the Telegram bot
CMD ["python", "-m", "cryptotracker_bot"]
```

```
# /backend/Dockerfile

FROM python:3.13-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    gcc \
    postgresql-client \
    && rm -rf /var/lib/apt/lists/*

# Install Python dependencies
COPY pyproject.toml ./
RUN pip install --no-cache-dir -e .

COPY . .

EXPOSE 8000

CMD ["uvicorn", "cryptotracker.__main__:app", "--host", "0.0.0.0", "--port",
"8000"]
```

```
# /frontend/Dockerfile

# Build stage
```

FROM node:20-alpine AS builder

WORKDIR /app

Copy package files

COPY package.json package-lock.json ./

Install dependencies

RUN npm ci

Copy source code

COPY . .

Set build environment variables

ENV VITE_API_BASE_URL=/api/v1

Build the application

RUN npm run build

Production stage with nginx

FROM nginx:alpine

Copy nginx config

COPY nginx.conf /etc/nginx/nginx.conf

Copy built files from builder

COPY --from=builder /app/dist /usr/share/nginx/html

Expose port

EXPOSE 3001

Start nginx

CMD ["nginx", "-g", "daemon off;"]

docker-compose.yml

version: '3.8'

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: crypto-tracker-db
    environment:
      POSTGRES_DB: cryptotracker_db
      POSTGRES_USER: user
      POSTGRES_PASSWORD: hackme
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user"]
      interval: 10s
      timeout: 5s
      retries: 5

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: crypto-tracker-backend
    environment:
      POSTGRES_DB: cryptotracker_db
      POSTGRES_HOST: postgres
      POSTGRES_USER: user
      POSTGRES_PASSWORD: hackme
      POSTGRES_PORT: 5432
    ports:
      - "8080:8000"
    depends_on:
      postgres:
        condition: service_healthy
    volumes:
      - ./backend:/app
    command: uvicorn cryptotracker.__main__:app --host 0.0.0.0 --port 8000
0 --reload
```

```
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  container_name: crypto-tracker-frontend
  ports:
    - "3001:3001"
  depends_on:
    - backend
  networks:
    - default

bot:
  build:
    context: ./bot
    dockerfile: Dockerfile
  container_name: crypto-tracker-bot
  env_file:
    - ./bot/.env
  environment:
    # Use the internal Docker network name for the backend
    API_BASE_URL: http://backend:8000/api/v1
  depends_on:
    - backend

volumes:
  postgres_data:
```